

Calibration of VCG Training Loss and PenaltyQAOA Penalty Strength

Completed calibration results, 10 September 2026

1 Calibration of Training Loss and Penalty Strength

We conducted two calibration studies before the main algorithm comparison. Study A selected a VCG training loss for preparing the uniform superposition of feasible states. Study B selected a rule for setting the shared penalty coefficient in standard PenaltyQAOA. Study B did not use trained VCGs and did not execute PC-QAOA. The studies used separate instance sets and selection criteria. The calibration COPs are reserved from the subsequent algorithm comparison.

1.1 Study A: VCG Loss Selection

Constraint set. We generated 50 constraints on support sizes $k \in \{4, 5, 6, 7, 8\}$, with ten constraints per size and two or three per family within each size. The family totals were 13 positive weighted inequalities, 13 mixed-sign inequalities, 12 weighted equalities, and 12 quadratic inequalities. For positive weighted inequalities, coefficients were integers in $\{1, \dots, 5\}$. If S denotes their sum, the right-hand side was sampled uniformly from the integers between $\max(1, \lfloor S/3 \rfloor)$ and $\lfloor 2S/3 \rfloor$, inclusive. Quadratic inequalities used dense upper-triangular coefficient arrays, including diagonal terms, with coefficients in $\{1, 2, 3\}$ and the same capacity rule applied to the coefficient sum. Diagonal terms satisfy $x_i^2 = x_i$ on binary inputs.

Mixed-sign inequalities used coefficient magnitudes in $\{1, \dots, 5\}$ and required both signs. Writing a and b for the minimum and maximum attainable left-hand-side values, the integer right-hand side was drawn from $[a + \lfloor (b - a)/3 \rfloor, a + \lfloor 2(b - a)/3 \rfloor]$. Weighted equalities used positive coefficients in $\{1, \dots, 5\}$ and an attainable right-hand side in the positive-weight capacity range. We rejected empty and fully feasible sets, repeated feasible masks at the same support size, and cases supported by the existing exact-preparation routines, including singleton sets and unions of complete Hamming-weight sectors. The accepted feasible fractions (the total amount of bitstrings feasible out of all 2^k bitstrings) ranged from 0.078125 to 0.90625 (Fig. 1).

Loss functions. For feasible set $\mathcal{F}$, the target state and evaluation metrics were

$$|u_{\mathcal{F}}\rangle = |\mathcal{F}|^{-1/2} \sum_{x \in \mathcal{F}} |x\rangle, \quad (1)$$

$$P_{\mathcal{F}}(\theta) = \sum_{x \in \mathcal{F}} |\langle x | \psi_{\theta} \rangle|^2, \quad F(\theta) = |\langle u_{\mathcal{F}} | \psi_{\theta} \rangle|^2, \quad (2)$$

$$F_{\text{cond}}(\theta) = F(\theta)/P_{\mathcal{F}}(\theta). \quad (3)$$

We compared five candidate loss functions, which had the form:

$$L_{\text{feas}} = 1 - 2P_{\mathcal{F}}, \quad L_{\text{fid}} = 1 - F, \quad (4)$$

$$L_{\lambda} = (1 - P_{\mathcal{F}}) + \lambda(1 - F_{\text{cond}}), \quad \lambda \in \{0.5, 1, 2\}. \quad (5)$$

Every loss was applied to every constraint, giving 250 complete VCG training tasks (5 losses $\times$ 50 constraints). Each task used one complete training seed and the restart budgets in Table 1. Initialization seeds were matched across losses for each constraint and depth to compare performance fairly. The inherited optimized angles could differ between losses.

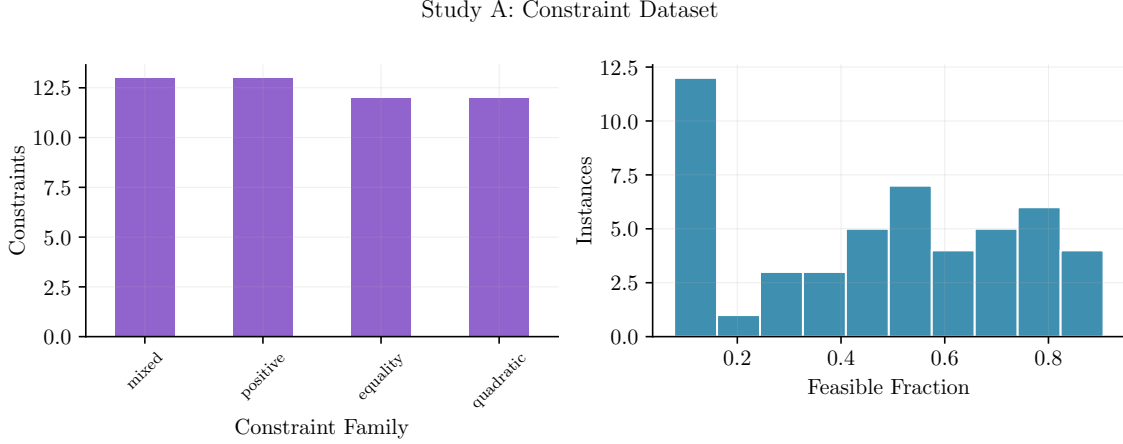


Figure 1: Study A constraint composition and feasible fractions. A feasible fraction is the number of satisfying bitstrings divided by 2^k .

Table 1: Training budgets. A restart runs the full stated number of Adam steps. Early stopping in Study A occurs between depths, not within a restart.

Setting	Study A	Study B
Standard-QAOA warm-up	$p = 1, 5 \times 150$ steps	None
Main angle strategy	ma-QAOA	Standard QAOA
Depths	1–8	1–5
Restarts per main depth	20	10
Adam steps per restart	200	50
Learning rate	0.05	0.01
Final measurements per depth	Exact state metrics	10,000 shots

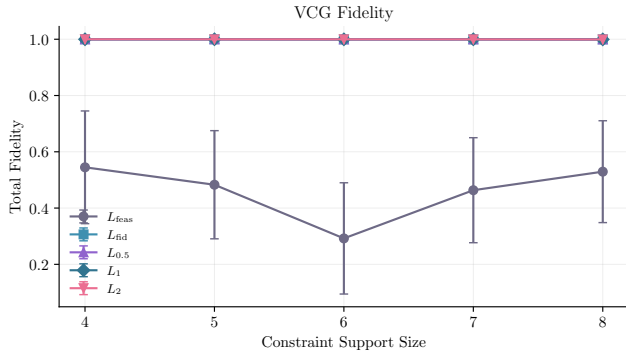
Training and retention. Training used the same warm-starting procedure we used in the manuscript; we retained the best angles at the end of each layer of ma-QAOA to initialize subsequent layers of optimization. At each depth, the restart with the lowest final value of its assigned training loss was retained. Fidelity was then evaluated for that state. Training stopped if this fidelity reached 0.999, or after depth eight. The task returned the highest-fidelity state among the retained depth states. Tasks that failed to reach the threshold remained in the analysis. These fidelity-based stopping and return rules replaced the entropy-based rules of the earlier experimental workflow.

Results. The selected loss was L_{fid} , which achieved mean fidelity 0.999719 and reached $F \geq 0.999$ on all 50 constraints (Table 2). Its mean selected depth was 3.18, compared with 3.42–3.62 for the conditional-fidelity losses. Each conditional-fidelity loss reached the threshold on 49 constraints. Their mean fidelities were also high, and their small differences from L_{fid} do not establish a broad performance separation. We should choose L_{fid} for our loss function.

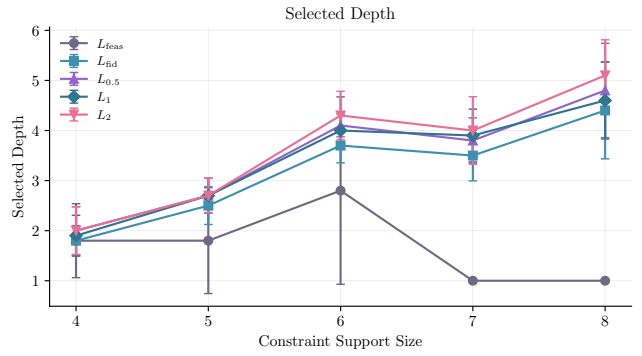
The feasibility loss (the one we used in the original manuscript) achieved mean feasible probability 0.994615, but mean total fidelity was only 0.462554 and only two constraints reached the fidelity threshold. Its mean selected depth was 1.68, although it evaluated 7.72 depths on average. Figure 3 shows the difference between feasibility and fidelity; Fig. 4 compares the probability distributions for one constraint under two losses. These results support including total fidelity in the training objective when the desired gadget prepares the uniform feasible superposition.

Table 2: Study A results across all 50 constraints per loss. Depth columns report the mean selected depth and the mean number of evaluated ma-QAOA depths.

Loss	Mean F	95% CI	Mean $P_{\mathcal{F}}$	$F \geq 0.999$	Selected p	Evaluated p
L_{fid}	0.999719	[0.999636, 0.999802]	0.999849	50/50	3.18	3.18
L_1	0.999695	[0.999583, 0.999807]	0.999837	49/50	3.42	3.44
$L_{0.5}$	0.999672	[0.999550, 0.999794]	0.999912	49/50	3.48	3.48
L_2	0.999594	[0.999427, 0.999761]	0.999667	49/50	3.62	3.64
L_{feas}	0.462554	[0.385114, 0.539994]	0.994615	2/50	1.68	7.72



(a) Returned fidelity by support size.



(b) Selected depth by support size.

Figure 2: Study A means and pointwise 95% confidence intervals across constraints. Each support-size mean contains ten constraints. The selected depth is the depth of the returned gadget, not necessarily the final depth evaluated.

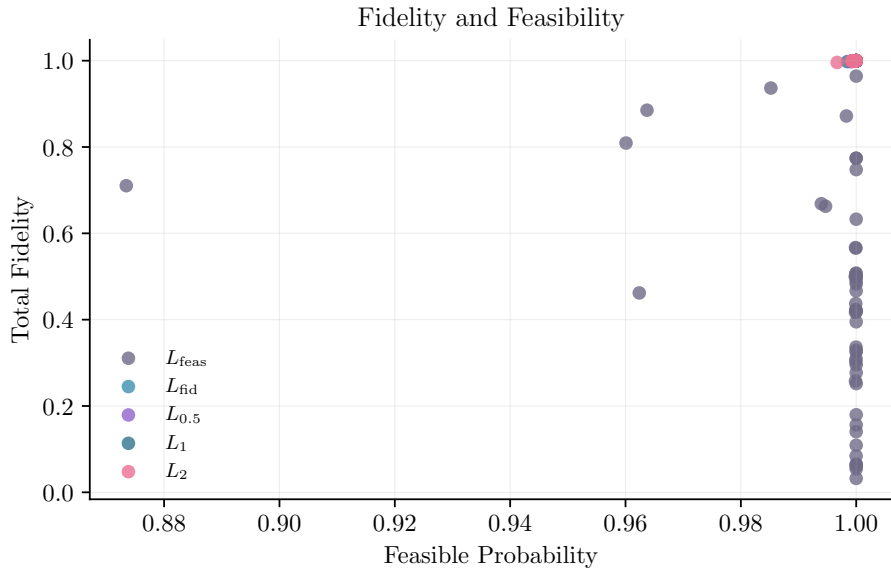


Figure 3: Returned-state fidelity versus feasible probability. Each point is one constraint–loss task.

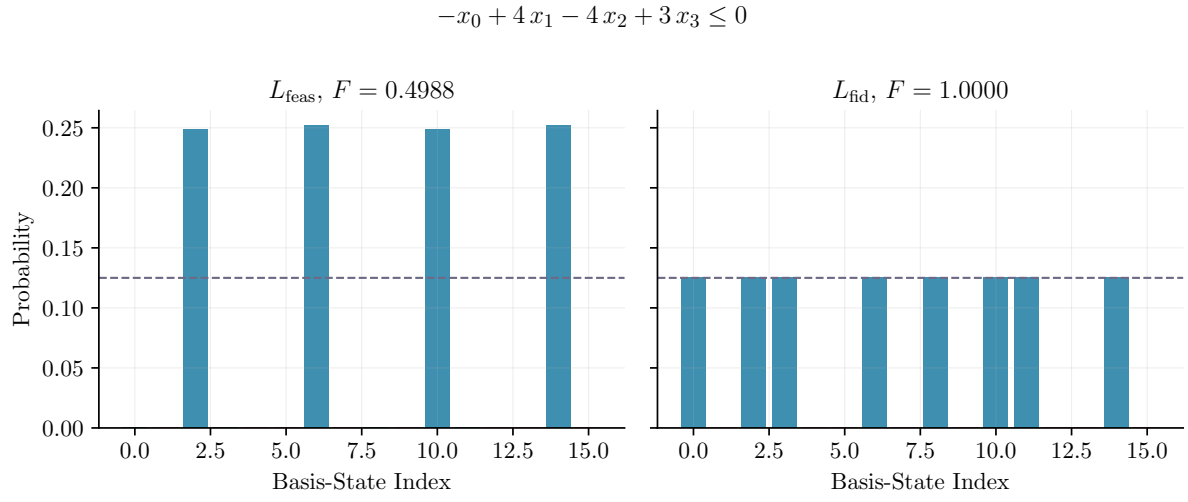


Figure 4: Returned probability distributions for the same mixed-sign constraint trained with L_{feas} and L_{fid} . Blue denotes feasible states and rose denotes infeasible states (note how there is no rose!). The dashed line is $1/|\mathcal{F}|$, the target probability on each feasible state. (Note how bad L_{feas} performs compared to L_{fid}).

1.2 Study B: Penalty-Rule Selection

The goal of this study is to determine the best way to select the penalty parameter δ used in PenaltyQAOA.

Calibration problems. We generated 20 fresh COPs with decision sizes $n \in \{4, 5, 6, 7, 8\}$. Each size contributed two problems with disjoint constraint supports and two with overlapping supports. Across the 50 constraint occurrences, there were 12 cardinality, 10 knapsack, nine quadratic-knapsack, eight flow, six independent-set, and five assignment constraints. The constraints used in this experiment were different from those generated in the VCG training experiment.

Each objective was generated independently as an upper-triangular QUBO with integer coefficients from -5 through 4 . The COPs were checked for feasibility and repeated constraint collections were rejected. Joint feasible fractions ranged from 0.0625 to 0.75 (Fig. 5). The complete dataset was saved before training.

Five penalty rules. Write the objective with each coefficient counted once:

$$f(x) = \sum_i q_i x_i + \sum_{i < j} q_{ij} x_i x_j. \quad (6)$$

For $i \neq j$, define $c_{ij} = q_{\min(i,j), \max(i,j)}$. Let q_t range over the linear and pairwise coefficients, and set

$$L_Q = \sum_t \min(0, q_t), \quad U_Q = \sum_t \max(0, q_t). \quad (7)$$

The five candidates were

$$\delta_{\text{range}} = U_Q - L_Q + 1, \quad (8)$$

$$\delta_{\text{feasible}} = f(\bar{x}) - L_Q + 1, \quad (9)$$

$$\delta_{\text{VL}} = 1 + \max_i \left\{ q_i + \sum_{j \neq i} \max(c_{ij}, 0), -q_i - \sum_{j \neq i} \min(c_{ij}, 0) \right\}, \quad (10)$$

$$\delta_{\text{local}} = 1 + \max_i \left(|q_i| + \sum_{j \neq i} |c_{ij}| \right), \quad (11)$$

$$\delta_{\text{max}} = 1 + \max_t |q_t|. \quad (12)$$

For each of the 20 COPs, we tested all five δ options, leading to 100 total optimized COPs.

The feasible point $\bar{x}$ was the first feasible string encountered in a seeded random ordering of decision strings, searched without replacement. No objective values or optimal-solution labels entered that search. The objective was evaluated after the point was chosen. The search required a mean of 3.2 feasibility checks, with a maximum of 11; its measured mean runtime was approximately 0.149 ms on this machine. This search is inexpensive for the calibration sizes, but its enumeration-based implementation is not a scalable feasibility algorithm.

The rules δ_{range} and δ_{feasible} are sufficient for the implemented integer squared-residual encoding. To see this, write the penalized energy as $f(x) + \delta \sum_j r_j(x, s_j)^2$. For an infeasible decision string, its minimum residual sum over slack assignments is at least one. With δ_{feasible} , its energy is therefore at least $L_Q + \delta_{\text{feasible}} = f(\bar{x}) + 1$, above the energy of a feasible point with valid slack. The bound for δ_{range} follows by replacing $f(\bar{x})$ with its upper bound U_Q . These guarantees concern global energy minima, not the success probability of finite-depth QAOA.

The signed local rule δ_{VL} uses the objective-change calculation of Verma and Lewis [1], with an added unit margin. The rules δ_{VL} , δ_{local} and δ_{max} are heuristics for the general overlapping constraints considered here. A one-bit objective-change bound alone does not prove that arbitrary infeasible assignments can be repaired with a sufficient decrease in penalty. The sufficient-bound reasoning is consistent with the exact-penalty framework discussed by Diez García et al. [2]; their solver experiments do not establish QAOA performance for these instances.

Optimization and measurements. We optimized using standard QAOA with the hyperparameters in Table 1, following the same warm-starting procedure in the manuscript once again. At each depth, the restart with the lowest final penalized expectation was retained. Final performance was determined by calculating $P(\text{opt})$ from 10,000 measurement shots.

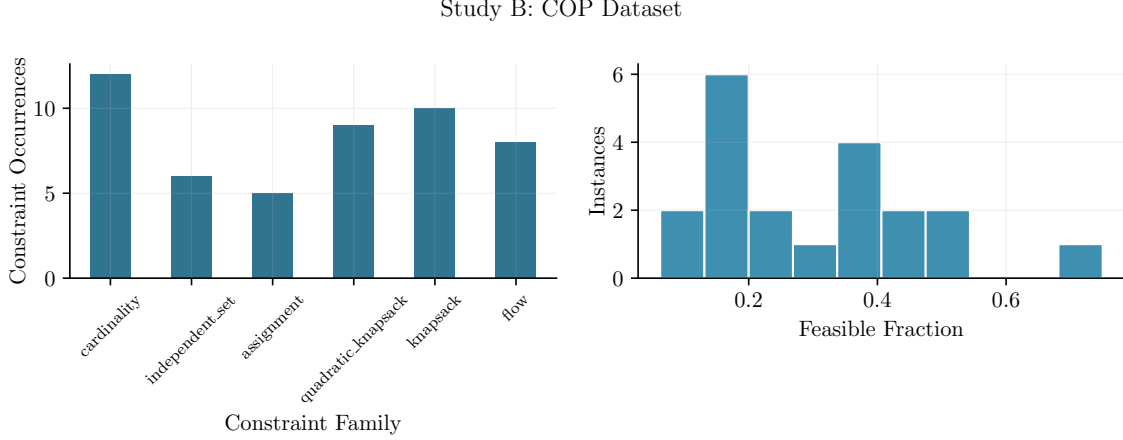


Figure 5: Study B constraint occurrences and joint feasible fractions. Each feasible fraction counts decision strings satisfying all constraints simultaneously, divided by 2^n .

Table 3: Study B results at depth five of QAOA, with 20 COPs per method. Probabilities and interval bounds are percentages.

Method	Sampled $P(\text{opt})$	95% CI	Exact $P(\text{opt})$	Sampled $P(\mathcal{F})$
δ_{range}	12.377	[1.418, 23.337]	12.403	46.127
δ_{max}	11.284	[2.018, 20.550]	11.329	43.681
δ_{VL}	10.920	[2.050, 19.790]	10.856	46.550
δ_{local}	10.736	[1.265, 20.206]	10.827	44.846
δ_{feasible}	9.878	[0.550, 19.207]	9.783	45.449

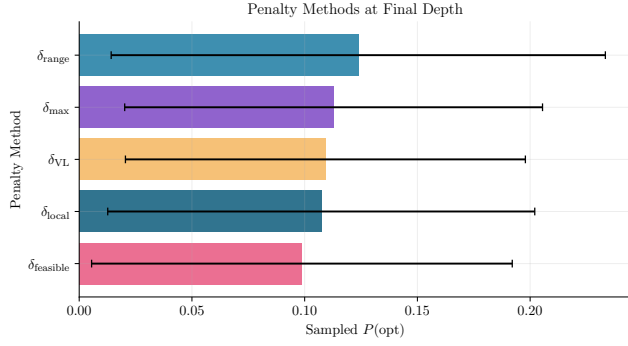
Results and selection. δ_{range} achieved the highest mean sampled $P(\text{opt})$, 12.3775%, followed by δ_{max} at 11.2840% (Table 3). The ordering of the mean exact probabilities was the same. We should therefore select δ_{range} in Eq. (8) for the subsequent penalty-based experiments when we do the full PC-QAOA vs. PenaltyQAOA experiment. Figure 6 reports the final-depth means and the full depth progression.

Performance varied strongly with the sampled instances. For δ_{range} , mean $P(\text{opt})$ was 40.588% on the four-variable COPs and 0.615% on the eight-variable COPs (Table 4). Its mean was 21.116% for disjoint COPs and 3.639% for overlapping COPs. These groups also differ in constraint composition and slack count; the differences cannot be assigned to size or overlap alone. The two four-variable disjoint COPs contributed substantially to the mean advantage of δ_{range} . δ_{range} remained first after removing any single COP, but removing both of those COPs changed the leader to δ_{max} . That post hoc sensitivity check did not change the selection dataset or criterion.

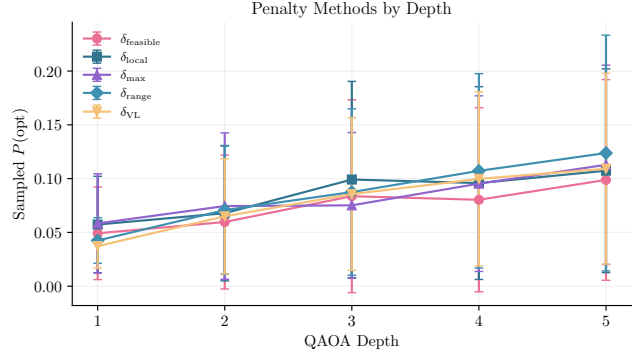
The completed calibration supports L_{fid} as a direct and effective training objective for the sampled VCG families. It selects δ_{range} as the penalty rule with the highest aggregate sampled success probability at depth five.

Table 4: Mean sampled depth-five $P(\text{opt})$ (percent) by decision size, with four COPs per size. Different methods lead at different sizes.

n	δ_{range}	δ_{feasible}	δ_{VL}	δ_{local}	δ_{max}
4	40.587	34.025	31.677	32.903	33.428
5	11.045	3.620	11.220	10.935	14.095
6	7.940	10.237	9.605	7.255	6.582
7	1.700	1.020	0.860	1.710	1.650
8	0.615	0.490	1.238	0.875	0.665



(a) Final-depth method comparison.



(b) Depth progression.

Figure 6: Study B mean sampled optimal-solution probabilities and pointwise 95% confidence intervals across 20 COPs. These intervals estimate uncertainty in each method’s mean; they are not paired intervals for differences between methods.

It does not show that this rule is optimal at other depths or for each constraint family.

A Implementation and Reproducibility

The two studies used the existing repository generators, constraint parsing, VCG circuits, and PenaltyQAOA implementation through the tuning scripts. Computation used PennyLane 0.43.1 with the `default.qubit` simulator, JAX/JAXlib 0.6.2, Optax 0.2.5, NumPy 2.2.6, and Python 3.13.13, with 64-bit CPU arithmetic. The dataset seed was 42, the training seed root was 7, and the sampling seed root was 19. Stable hashes of instance and phase identifiers derived the optimization seeds. Penalty-setting identifiers additionally entered the sampling seeds. No complete training-seed replication was performed.

Tasks were grouped by constraint or COP to reuse circuit preparation and compiled functions across conditions. Standard-QAOA penalty circuits combined repeated commuting Pauli terms before compilation while retaining separate numerical objective and penalty weights. The shared angle permits this combination without changing the ideal circuit, apart from irrelevant identity phases. VCG circuits retained the existing ma-QAOA parameter layout. Optimization checkpoints stored angles, Adam state, random state and progress every ten steps and at restart, depth and task boundaries. Files were written atomically with a previous-checkpoint backup.

The default allocation was two workers with six physical CPU cores per worker. Accumulated compile/warm-up time was approximately 10.87 worker-hours for Study A and 0.57 worker-hours for Study B; synchronized optimizer-update time was approximately 10.22 and 14.18 worker-hours, respectively. These totals are sums across tasks, not elapsed wall time, and exclude some I/O and analysis costs. Compilation counters include disposable warm-up calls and overhead on reused functions. The largest recorded worker RSS was approximately 14.54 GiB in Study A and 14.82 GiB in Study B. A worker’s RSS is a process high-water mark, not a per-task allocation.

PenaltyQAOA used 4–19 total qubits, including 0–12 slack qubits. Figure 7 shows the measured simulation times and qubit counts. The displayed PC-QAOA slack counts are computed partitions only; no PC-QAOA simulation is included.

Archived evidence. The report’s `data/` directory contains copies of the original run records, manifests and selected-setting records, along with per-task summaries, ranking tables, and paired comparisons. The copied run records preserve source-file hashes, base commit and package versions. Study A and Study B have separate run identifiers because Study B was revised after Study A began. The obsolete multiplier configuration in Study A’s run record was not used for the reported Study B results; the separate Study B record specifies the five fixed methods. The report generation record also hashes the analysis source used to produce its tables and figures.

The selected VCG database contains one gadget per calibration constraint trained with L_{fid} . It remains in the original results directory. The report summarizes saved final results and does not modify selections, checkpoints, gadget states or optimizer histories.

B Additional Diagnostic Figures

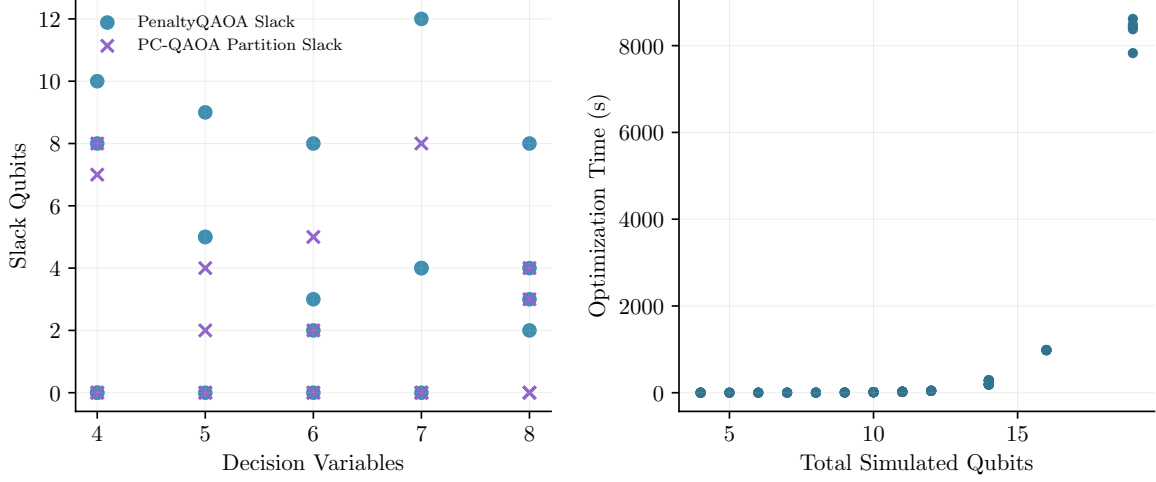
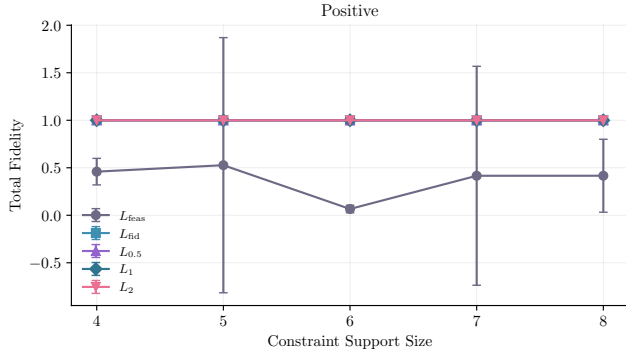
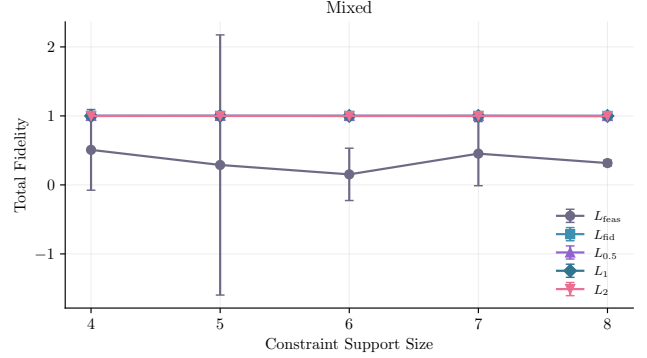


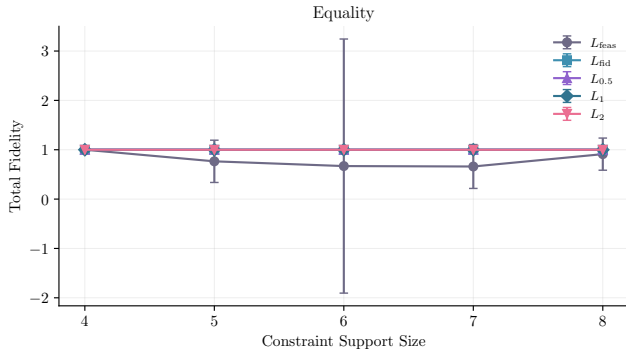
Figure 7: Decision/slack counts and recorded PenaltyQAOA optimizer-update times. Each runtime point is a completed instance–method task across depths one through five. The PC-QAOA points represent hypothetical partition slack counts, not measured PC-QAOA execution.



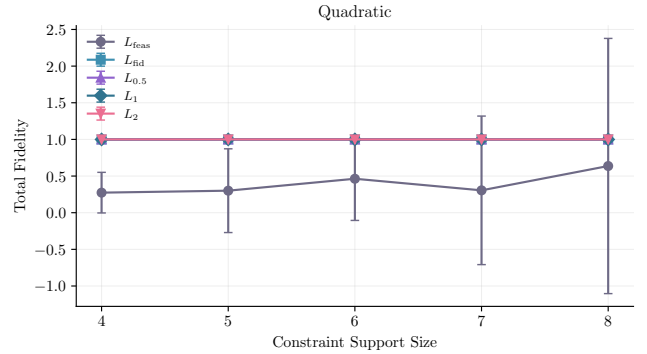
(a) Positive weighted inequalities.



(b) Mixed-sign inequalities.



(c) Weighted equalities.



(d) Quadratic inequalities.

Figure 8: Returned VCG fidelity by family and support size. Each point contains two or three constraints. The pointwise t intervals can be wide or extend beyond physical probability bounds because of the small groups.

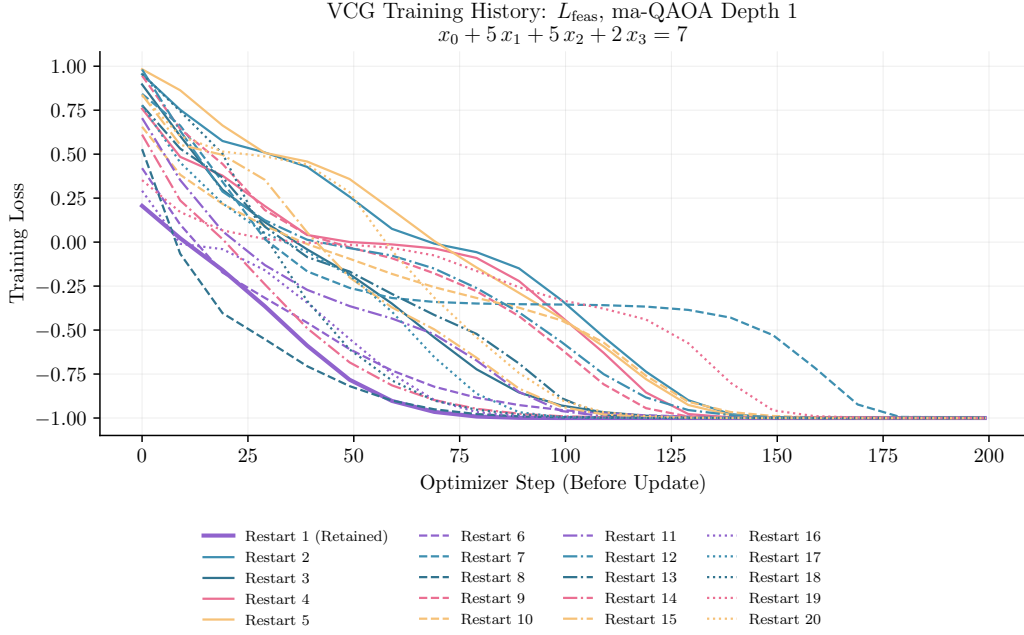
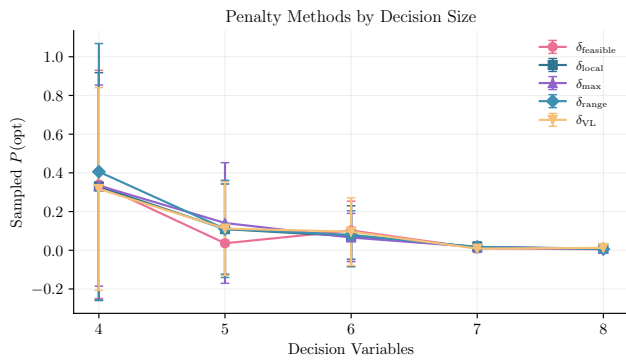
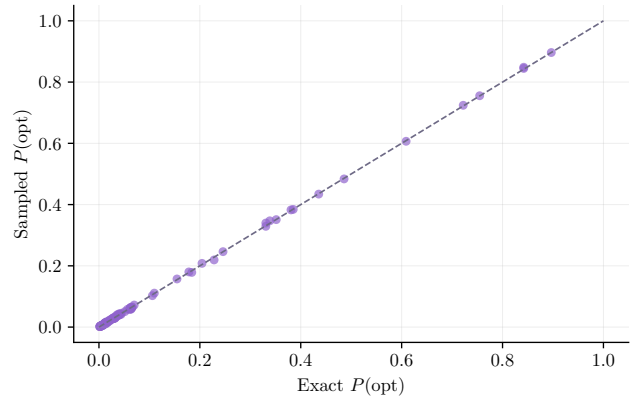


Figure 9: Example VCG training histories. The title identifies the constraint, loss and depth. Each curve is one restart; the retained restart has the lowest final training loss. The plotted loss is evaluated before the indicated update, whereas restart selection uses the loss after the final update. Loss approaching -1 under L_{feas} indicates feasible probability approaching one, not necessarily uniform-state fidelity approaching one.



(a) Size dependence.



(b) Sampled and exact probabilities.

Figure 10: Study B diagnostics. Left: means over four COPs per decision size, with pointwise 95% intervals. Right: exact and sampled depth-five success probabilities for all 100 tasks. The diagonal denotes equality; exact values do not enter penalty selection.

References

- [1] A. Verma and M. Lewis, “Penalty and partitioning techniques to improve performance of QUBO solvers,” *Discrete Optimization* **44**, 100594 (2022). [doi:10.1016/j.disopt.2020.100594](https://doi.org/10.1016/j.disopt.2020.100594).
- [2] M. Diez García, M. Ayodele, and A. Moraglio, “Exact and Sequential Penalty Weights in Quadratic Unconstrained Binary Optimisation with a Digital Annealer,” *GECCO ’22 Companion*, pp. 184–187 (2022). [doi:10.1145/3520304.3528925](https://doi.org/10.1145/3520304.3528925).